Noted!

Write music one note at a time and see it as real sheet music

Milestone 1.0 · Team Hinton · IST 303 · Nov 12, 2026

Who presents what

1	**Varuzhan**	What Noted! does and how we worked (agile + burndown)	4 min
2	**Daniel**	Live demo, part 1: account, library, create, add notes and rests	4 min
3	**Jae**	Live demo, part 2: sheet music, select, edit and delete a note, rename a piece	4 min
4	**Ryan**	How we tested the code	4 min
5	**Aaron**	Successes, failures, lessons learned, Milestone 2.0	4 min

What Noted! does

Notation apps like Sibelius and Finale are expensive and slow to learn. Quick options like staff paper or voice memos are hard to edit. Noted! lets you enter a melody note by note and see it as sheet music right away.

Your own account

Register, log in, and every piece stays private

Write note by note

Pitch, octave, length, accidental, or a rest

See real sheet music

Clef, key and time signature, bar lines

Fix mistakes

Edit or delete notes. Rename or delete a piece

My Library

Your pieces, newest edit first

Follows the key

F in G major is drawn as F♯ automatically

How we worked



Two 3-week sprints

Stories split into tasks with hour estimates, owners and due dates on our Jira board



Weekly stand-ups

Wednesday nights: status, blockers, decisions. Notes in meetings/



One PR per task

Branch named NTT-##, filled-in description, a reviewer, automatic tests



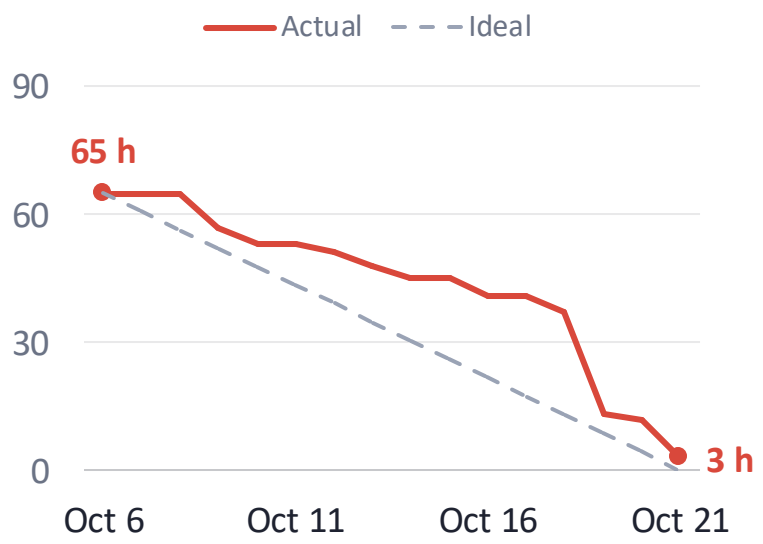
Retro after Sprint 1

Velocity 0.42, not 0.7. In Sprint 2 we made dependencies visible and added auto tests

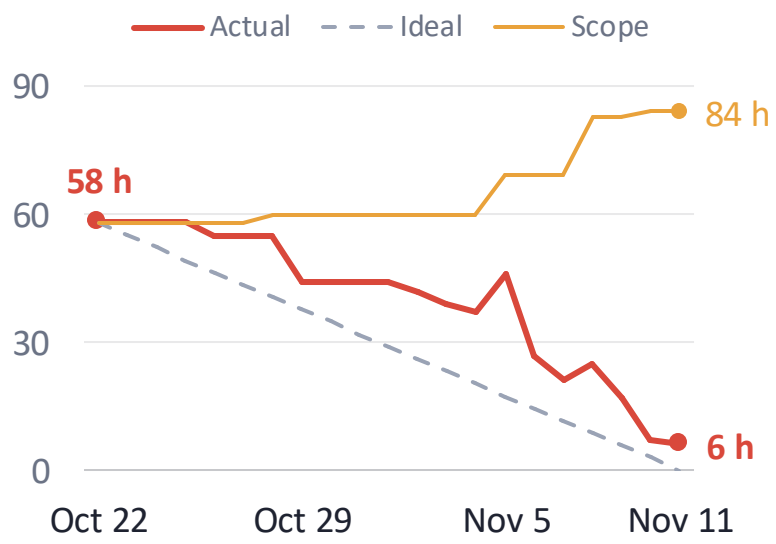
When Aaron was out sick for a week, his tasks were split between 3 teammates the same day, & Sprint 2 still finished all its features.

Burndown: both iterations

Sprint 1 hours remaining, Oct 6–21



Sprint 2 hours remaining, Oct 22–Nov 11



Sprint 1: **44** of **47** hours done. The **drop** on **Oct 19** is **US-04** moving to **Sprint 2**.
Sprint 2: **78** of **84** hours done by **Nov 11**; all **code done**. The **line jumps** because we **expanded our scope**.

Live demo, part 1

1	Register or log in	US-01, US-02
2	Open My Library	US-10
3	Create a piece: title, key, time, tempo	US-03
4	Add notes with the palette	US-04
5	Add a rest	US-05

Live demo, part 2



See it as sheet music

US-06

Key signature, rests, bar lines, drawn after every change



Select and edit a note

US-07

Click it on the staff or in the list; the palette fills in



Delete a note

US-08

Confirm, and the rest renumber with no gaps

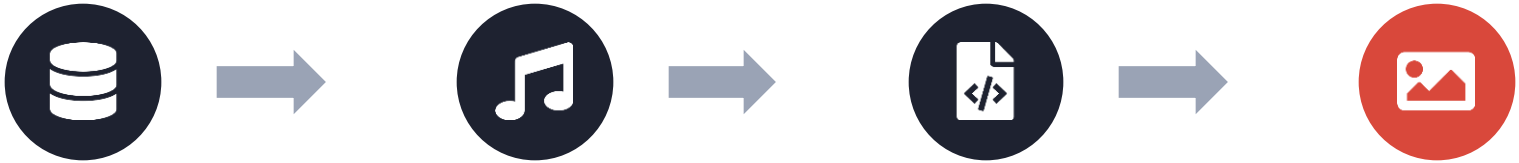


Rename a piece

US-09

Edit details → change the title → Save

How the sheet music is drawn



Saved notes

SQLite, in order

music21

Notes become a Stream

MusicXML

Standard music format

Verovio

Draws it as SVG

We picked this in a Sprint 1 spike instead of drawing the staff ourselves. That cut the sheet-music story from 17 to 10 hours. Each drawn note keeps its database id, so clicking it on the staff selects that note.

How we tested

555

tests passing

98%

app code coverage

Every PR

tests run automatically (CI)

- pytest with shared fixtures and an in-memory database
- Tests written from each story's acceptance criteria (e.g. test_US_06_T5.py for sheet music)
- Tests run by GitHub on every pull request
- Privacy tests: every page returns ‘not found’ for another user’s piece (US-12)

A test caught a real bug: an out-of-range tempo was rejected, but the error message never appeared on the page. Daniel’s US-09 test found it, and he fixed it in the same PR (#60)

What went well, what didn't



Went well

- Early rendering spike helped us choose the right tools.
- Automated tests helped us check changes before merging.
- Teammates supported each other through PR reviews.



Didn't go well

- Some PR reviews took longer than expected.
- Outdated branches caused merge conflicts.
- Some Sprint 2 tests had to wait for related PRs to merge.

Three lessons we learned



Lesson 1: Test risky ideas early

Our rendering spike helped us choose music21 and Verovio before implementation.



Lesson 2: Review PRs quickly and communicate clearly

Delayed reviews and merge conflicts showed us why 24-hour reviews and clear dependencies matter.



Lesson 3: Testing makes teamwork more reliable

Tests for note editing and deletion helped us verify changes without breaking existing features.

Milestone 2.0: 9 weeks, 3 iterations

3 Iteration 3

Weeks 1–3 · 40 h

- Library search
- PDF export
- Additional clefs
- Composition sharing
- Milestone 1.0 follow-ups

4 Iteration 4

Weeks 4–6 · 41 h

- Music playback
- Transcription prototype
- Audio upload and pitch tracking

5 Iteration 5

Weeks 7–9 · 49 h

- Complete audio transcription
- Practice mode for teachers

Planned at 0.4, a little below our measured velocity, because the audio work is the riskiest



Thank you

Questions?

github.com/team-hinton-ist303/ist-303-team-hinton